

IUT d'Aix-Marseille – Département Informatique

BUT Informatique – 2^e année

R3.07 – Bases de données

TD2 : découverte de PL/pgSQL

Travail sur feuille

Base support : `mini-pagila` (identique au TD1)

Version étudiants

4 septembre 2026

Déroulement de la séance

SQL sait exprimer « ce que l'on veut », mais pas « comment y arriver pas à pas » : pas de variable, pas de test, pas de boucle. **PL/pgSQL** est le langage procédural de PostgreSQL ; il permet d'écrire, *dans la base*, des fonctions qui alternent instructions SQL et logique algorithmique. C'est la première fois que vous le rencontrez : cette séance vise donc la **familiarisation avec la syntaxe** et avec les quelques mécanismes qui n'ont pas d'équivalent dans les langages que vous connaissez déjà.

La séance se déroule **sur feuille**, sans SGBD. Comme au TD1, l'enjeu est de *prédire* et de *justifier* le comportement d'un programme plutôt que de l'essayer jusqu'à ce qu'il fonctionne – une compétence d'autant plus utile en PL/pgSQL que beaucoup d'erreurs y sont *silencieuses* : le programme ne plante pas, il renvoie simplement une valeur fausse.

Deux ressources sont à garder sous les yeux pendant toute la séance :

- l'**annexe A** du TD2 (page 12) : aide-mémoire de la syntaxe PL/pgSQL. Tout ce qui est nécessaire pour traiter le sujet s'y trouve ; rien n'est à connaître par cœur aujourd'hui.
- l'**annexe A** du TD1 : schéma et contenu de la base **mini-pagila**, inchangée depuis le TD1.

Le sujet comporte deux sections :

Section 1 – programmes « de base » : déclarations, affectations, conditions, boucles, formes de retour et exceptions ;

Section 2 – curseurs (implicites et explicites) et programmes plus avancés.

Chaque section reprend les cinq types d'exercices du TD1 : lire, juger un résultat, compléter, corriger, écrire – plus une analyse critique d'un programme généré par une IA. Les questions marquées d'une étoile (*) sont des approfondissements.

Les six pièges de la séance

Ces six comportements expliquent la quasi-totalité des erreurs que vous ferez aujourd'hui. Ils sont détaillés dans l'annexe A, et chacun est travaillé au moins une fois dans le sujet.

- P1.** `SELECT ... INTO` sans `STRICT` **ne lève jamais d'erreur** : s'il n'y a aucune ligne, la variable devient `NULL` ; s'il y en a plusieurs, la première arrivée est prise (et l'ordre n'est pas garanti).
- P2.** Une variable non initialisée vaut `NULL`, et `NULL` contamine tout : `NULL + 5` vaut `NULL`, `NULL || 'texte'` vaut `NULL`.
- P3.** `FOUND` est mis à jour par la *dernière* requête exécutée. Après un `SELECT COUNT(*) INTO ...`, il vaut *toujours vrai* : une requête d'agrégation renvoie toujours une ligne.
- P4.** `IF v = NULL` n'est jamais vrai. On écrit `IF v IS NULL`.
- P5.** Après un `FETCH` qui échoue, la variable réceptrice est mise à `NULL` : continuer à l'utiliser ne provoque pas d'erreur, mais propage des `NULL`. Il faut utiliser `EXIT WHEN NOT FOUND` juste après le `FETCH` pour éviter cela.
- P6.** Une fonction qui atteint son `END` sans `RETURN` provoque une erreur à l'exécution – et un `RAISE NOTICE` n'est *pas* une valeur de retour.

1 Programmes de base

Exercice 1.1 – Lire un programme (PL/pgSQL → français)

Pour chacune des fonctions suivantes : (i) donnez-lui un nom parlant et rédigez en une ou deux phrases l'énoncé auquel elle répond ; (ii) donnez le résultat des appels indiqués, sur l'instance de la base `pagila` vue en TD1.

- a) Appels : `mystere_a(6)`, `mystere_a(9)`, `mystere_a(2)`, `mystere_a(99)`.

```
CREATE OR REPLACE FUNCTION mystere_a(p_film_id INTEGER) RETURNS TEXT AS $$
DECLARE
    v_titre film.title%TYPE;
    v_duree film.length%TYPE;
BEGIN
    SELECT title, length INTO v_titre, v_duree FROM film WHERE film_id = p_film_id;
    IF NOT FOUND THEN
        RETURN 'Film inconnu';
    END IF;
    IF v_duree IS NULL THEN
        RETURN v_titre || ' (durée inconnue)';
    ELSIF v_duree < 90 THEN
        RETURN v_titre || ' (court)';
    ELSE
        RETURN v_titre || ' (long)';
    END IF;
END;
$$ LANGUAGE plpgsql;
```

- b) Appels : `mystere_b(6)`, `mystere_b(8)`, `mystere_b(99)`.

```
CREATE OR REPLACE FUNCTION mystere_b(p_customer_id INTEGER) RETURNS TEXT AS $$
DECLARE
    v_nom customer.last_name%TYPE;
    v_nb INTEGER;
    v_res TEXT := '';
BEGIN
    SELECT last_name INTO v_nom FROM customer WHERE customer_id = p_customer_id;
    SELECT COUNT(*) INTO v_nb FROM rental WHERE customer_id = p_customer_id;
    FOR i IN 1..v_nb LOOP
        v_res := v_res || '*';
    END LOOP;
    RETURN v_nom || ' : ' || v_res;
END;
$$ LANGUAGE plpgsql;
```

- c) Appels : `mystere_c(5)`, `mystere_c(3)`, `mystere_c(0.5)`.

```
CREATE OR REPLACE FUNCTION mystere_c(p_budget NUMERIC) RETURNS INTEGER AS $$
DECLARE
    v_tarif NUMERIC;
    v_reste NUMERIC := p_budget;
    v_nb INTEGER := 0;
BEGIN
    SELECT MIN(rental_rate) INTO v_tarif FROM film;
    WHILE v_reste >= v_tarif LOOP
        v_reste := v_reste - v_tarif;
        v_nb := v_nb + 1;
    END LOOP;
    RETURN v_nb;
END;
```

```
|| $$ LANGUAGE plpgsql;
```

Exercice 1.2 – Ce résultat est-il cohérent ?

On fournit une fonction et une série d'appels avec le résultat *prétendument* obtenu. Pour chaque ligne : le résultat est-il cohérent ? Sinon, expliquez pourquoi et donnez le résultat correct.

Cas A

```
CREATE OR REPLACE FUNCTION prix_double(p_film_id INTEGER) RETURNS NUMERIC AS $$
DECLARE v_tarif NUMERIC;
BEGIN
    SELECT rental_rate INTO v_tarif FROM film WHERE film_id = p_film_id;
    RETURN v_tarif * 2;
END;
$$ LANGUAGE plpgsql;
```

Résultat proposé

Appel	Résultat annoncé
prix_double(3)	5.98
prix_double(99)	ERREUR : aucune ligne retournée

Cas B

```
CREATE OR REPLACE FUNCTION titre_au_tarif(p_tarif NUMERIC) RETURNS TEXT AS $$
DECLARE v_titre TEXT;
BEGIN
    SELECT title INTO STRICT v_titre FROM film WHERE rental_rate = p_tarif;
    RETURN v_titre;
END;
$$ LANGUAGE plpgsql;
```

Résultat proposé

Appel	Résultat annoncé
titre_au_tarif(1.99)	Island Empire
titre_au_tarif(4.99)	Bucket Brotherhood
titre_au_tarif(9.99)	NULL

Cas C

```
CREATE OR REPLACE FUNCTION taux_retour(p_customer_id INTEGER) RETURNS NUMERIC AS $$
DECLARE
    v_total    INTEGER;
    v_rendues  INTEGER;
BEGIN
    SELECT COUNT(*), COUNT(return_date) INTO v_total, v_rendues
    FROM rental WHERE customer_id = p_customer_id;
    RETURN ROUND(100.0 * v_rendues / v_total, 1);
EXCEPTION
    WHEN division_by_zero THEN
        RETURN NULL;
END;
$$ LANGUAGE plpgsql;
```

Résultat proposé

Appel	Résultat annoncé
taux_retour(2)	33.3
taux_retour(6)	75.0
taux_retour(8)	NULL

Exercice 1.3 – Compléter le programme

Complétez les trous numérotés. Un trou peut contenir plusieurs mots.

- a) *Renvoie le nombre de films d'une catégorie désignée par son nom, et lève une exception si la catégorie n'existe pas.*

```
CREATE OR REPLACE FUNCTION nb_films_categorie(p_nom TEXT) RETURNS INTEGER AS $$
DECLARE
    v_id .....(1).....;
    v_nb INTEGER;
BEGIN
    SELECT category_id .....(2)..... v_id FROM category WHERE name = p_nom;
    IF .....(3)..... THEN
        RAISE .....(4)..... 'Catégorie inconnue : %', p_nom;
    END IF;
    SELECT COUNT(*) INTO v_nb FROM film WHERE category_id = v_id;
    .....(5)..... v_nb;
END;
$$ LANGUAGE .....(6).....;
```

- b) *Renvoie la table des films d'une catégorie (titre et tarif), triée par titre.*

```
CREATE OR REPLACE FUNCTION films_de(p_nom TEXT)
RETURNS .....(1)..... (titre TEXT, tarif NUMERIC) AS $$
BEGIN
    RETURN .....(2).....
    SELECT f.title::TEXT, f.rental_rate
    FROM film f JOIN category ca ON f.category_id = ca.category_id
    WHERE ca.name = p_nom
    ORDER BY f.title;
END;
$$ LANGUAGE plpgsql;
```

Question complémentaire : comment appelle-t-on cette fonction, et en quoi cet appel diffère-t-il de celui de `nb_films_categorie` ?

Exercice 1.4 – Corriger le programme

Chacun des programmes suivants est faux. Indiquez **à quel moment** le problème se manifeste (au moment de la création de la fonction, ou seulement à l'appel ?), expliquez-le, et réécrivez la version correcte.

- a) *Renvoie la durée d'un film.*

```
CREATE OR REPLACE FUNCTION duree_film(p_id film.film_id%TYPE) RETURNS film.length%
TYPE AS $$
DECLARE v_duree film.length%TYPE;
BEGIN
    v_duree := SELECT length FROM film WHERE film_id = p_id;
    RETURN v_duree;
END;
$$ LANGUAGE plpgsql;
```

b) *Salue un client, ou signale qu'il n'existe pas.*

```
CREATE OR REPLACE FUNCTION bonjour_client(p_id customer.customer_id%TYPE) RETURNS
TEXT AS $$
DECLARE v_nom customer.last_name%TYPE;
BEGIN
    SELECT last_name INTO v_nom FROM customer WHERE customer_id = p_id;
    IF v_nom = NULL THEN
        RETURN 'Client inconnu';
    END IF
    RETURN 'Bonjour ' || v_nom;
END;
$$ LANGUAGE plpgsql;
```

c) *Renvoie le nom de la catégorie d'un film.*

```
CREATE OR REPLACE FUNCTION categorie_du_film(p_id film.film_id%TYPE) RETURNS
category.name%TYPE AS $$
DECLARE v_cat category.name%TYPE;
BEGIN
    SELECT ca.name INTO v_cat
    FROM film f JOIN category ca ON f.category_id = ca.category_id
    WHERE f.film_id = p_id;
    IF v_cat IS NOT NULL THEN
        RAISE NOTICE 'Catégorie : %', v_cat;
    END IF;
END;
$$ LANGUAGE plpgsql;
```

Exercice 1.5 – Écrire le programme

Écrivez les fonctions correspondant aux énoncés suivants. Indiquez à chaque fois comment on les appelle.

- a) `nb_locations(p_customer_id)` : renvoie le nombre de locations d'un client, et lève une exception explicite si le client n'existe pas.
- b) `classer_film(p_film_id)` : renvoie 'tous publics' pour un film classé G ou PG, 'accompagné' pour PG-13, 'adultes' pour R, et lève une exception si le film n'existe pas. Utilisez un CASE.
- c) `films_de_categorie(p_nom)` : renvoie la table (titre, durée) des films d'une catégorie, triée par titre.
- d) * `duree_moyenne_categorie(p_nom)` : renvoie la durée moyenne des films d'une catégorie, arrondie au dixième, et lève une exception si la catégorie n'existe pas. Que renvoie votre fonction pour la catégorie Documentaire ? Justifiez.

Exercice 1.6 – Regard critique sur un programme généré par IA

Énoncé soumis à l'assistant : « Écris une fonction PL/pgSQL qui renvoie le montant total des locations d'un client. Si le client n'existe pas, la fonction doit lever une exception. »

Programme proposé par une IA générative

```
CREATE OR REPLACE FUNCTION sum_payment_client(p_customer_id INTEGER)
RETURNS NUMERIC AS $$
DECLARE v_sum NUMERIC;
BEGIN
    SELECT SUM(amount) INTO v_sum FROM payment WHERE customer_id = p_customer_id;
```

```
IF NOT FOUND THEN
    RAISE EXCEPTION 'Client % inexistant', p_customer_id;
END IF;
RETURN v_sum;
END;
$$ LANGUAGE plpgsql;
```

- a) La fonction est-elle créée sans erreur ? Que renvoie-t-elle pour les appels (6), (8) et (99) ?
- b) Répond-elle à l'énoncé ? Justifiez en une phrase précise.
- c) Le client 8 (Paul Faure) existe mais n'a aucune location, et le client 99 n'existe pas. La fonction permet-elle de les distinguer ?
- d) Proposez une version correcte.

2 Curseurs et programmes avancés

Deux formes de parcours

Curseur implicite – la forme à privilégier quand on veut simplement traiter toutes les lignes :

```
FOR v_ligne IN SELECT ... LOOP          -- v_ligne doit être déclarée (RECORD)
...                                     -- ouverture, avancement et fermeture
END LOOP;                               -- sont gérés automatiquement
```

Curseur explicite – nécessaire quand on veut maîtriser l'avancement (s'arrêter en cours de route, lire une seule ligne, lire à un autre rythme) :

```
DECLARE c_x CURSOR FOR SELECT ... ;
BEGIN
  OPEN c_x;
  LOOP
    FETCH c_x INTO v_ligne;
    EXIT WHEN NOT FOUND;                -- immédiatement après le FETCH
    ...
  END LOOP;
  CLOSE c_x;
```

Exercice 2.1 – Lire un programme (PL/pgSQL → français)

Pour chacune des fonctions suivantes : (i) donnez-lui un nom parlant et rédigez en une ou deux phrases l'énoncé auquel elle répond ; (ii) donnez le résultat des appels indiqués ; (iii) précisez s'il s'agit d'un curseur implicite ou explicite.

- a) Appels : `mystere_d('Toulon')`, `mystere_d('Aix-en-Provence')`, `mystere_d('Lyon')`.

```
CREATE OR REPLACE FUNCTION mystere_d(p_ville customer.city%TYPE) RETURNS TEXT AS
$$
DECLARE
  v_ligne RECORD;
  v_res TEXT := '';
BEGIN
  FOR v_ligne IN
    SELECT c.last_name AS nom, COUNT(r.rental_id) AS nb
    FROM   customer c LEFT JOIN rental r ON c.customer_id = r.customer_id
    WHERE  c.city = p_ville
    GROUP BY c.customer_id, c.last_name
    ORDER BY nb DESC, c.last_name
  LOOP
    v_res := v_res || v_ligne.nom || '(' || v_ligne.nb || ') ';
  END LOOP;
  RETURN v_res;
END;
$$ LANGUAGE plpgsql;
```

- b) Appels : `mystere_e(5)`, `mystere_e(1)`, `mystere_e(100)`.

```
CREATE OR REPLACE FUNCTION mystere_e(p_budget NUMERIC) RETURNS INTEGER AS $$
DECLARE
  c_films CURSOR FOR SELECT title, rental_rate FROM film ORDER BY rental_rate,
  title;
  v_film RECORD;
  v_reste NUMERIC := p_budget;
  v_nb INTEGER := 0;
```

```

BEGIN
  OPEN c_films;
  LOOP
    FETCH c_films INTO v_film;
    EXIT WHEN NOT FOUND;
    EXIT WHEN v_film.rental_rate > v_reste;
    v_reste := v_reste - v_film.rental_rate;
    v_nb := v_nb + 1;
  END LOOP;
  CLOSE c_films;
  RETURN v_nb;
END;
$$ LANGUAGE plpgsql;

```

c) Appels : `SELECT * FROM mystere_f(5);` puis `SELECT * FROM mystere_f(8);`

```

CREATE OR REPLACE FUNCTION mystere_f(p_customer_id INTEGER) RETURNS SETOF TEXT AS
$$
DECLARE
  v_p      RECORD;
  v_cumul  NUMERIC := 0;
BEGIN
  FOR v_p IN SELECT payment_date, amount FROM payment
              WHERE customer_id = p_customer_id ORDER BY payment_date
  LOOP
    v_cumul := v_cumul + v_p.amount;
    RETURN NEXT v_p.payment_date || ' -> ' || v_cumul;
  END LOOP;
END;
$$ LANGUAGE plpgsql;

```

Exercice 2.2 – Ce résultat est-il cohérent ?

Cas A – la fonction est censée calculer le total payé par un client.

```

CREATE OR REPLACE FUNCTION total_paye(p_customer_id customer.customer_id%TYPE) RETURNS
  NUMERIC AS $$
DECLARE
  c_pay CURSOR FOR SELECT amount FROM payment WHERE customer_id = p_customer_id;
  v_p    RECORD;
  v_total NUMERIC := 0;
BEGIN
  OPEN c_pay;
  LOOP
    FETCH c_pay INTO v_p;
    v_total := v_total + v_p.amount;
    EXIT WHEN NOT FOUND;
  END LOOP;
  CLOSE c_pay;
  RETURN v_total;
END;
$$ LANGUAGE plpgsql;

```

Résultat proposé

Appel	Résultat annoncé
<code>total_paye(5)</code>	8.97
<code>total_paye(8)</code>	0.00

Cas B

```
CREATE OR REPLACE FUNCTION liste_longes(p_seuil INTEGER) RETURNS TEXT AS $$
DECLARE
    v_f RECORD;
    v_res TEXT := '';
BEGIN
    FOR v_f IN SELECT title, length FROM film WHERE length > p_seuil ORDER BY title LOOP
        v_res := v_res || v_f.title || ' ';
    END LOOP;
    IF v_res = '' THEN
        RETURN 'aucun';
    END IF;
    RETURN v_res;
END;
$$ LANGUAGE plpgsql;
```

Résultat proposé

Appel	Résultat annoncé
liste_longes(120)	Bucket Brotherhood; Escape Metropolis; Jungle Closer;
liste_longes(200)	aucun

Exercice 2.3 – Compléter le programme

La fonction parcourt les paiements d'un client par ordre chronologique et renvoie la date à laquelle le cumul de ses paiements dépasse un seuil donné, ou 'seuil jamais dépassé' le cas échéant.

```
CREATE OR REPLACE FUNCTION premier_depassement(p_customer_id payment.customer_id%TYPE,
    p_seuil NUMERIC)
RETURNS TEXT AS $$
DECLARE
    c_pay .....(1)..... FOR SELECT payment_date, amount FROM payment
                                WHERE customer_id = p_customer_id ORDER BY payment_date;
    v_p .....(2).....;
    v_cumul NUMERIC .....(3)..... 0;
BEGIN
    .....(4)..... c_pay;
    LOOP
        .....(5)..... c_pay INTO v_p;
        EXIT WHEN .....(6).....;
        v_cumul := v_cumul + v_p.amount;
        IF v_cumul > p_seuil THEN
            CLOSE c_pay;
            RETURN 'seuil dépassé le ' || v_p.payment_date;
        END IF;
    END LOOP;
    .....(7)..... c_pay;
    RETURN 'seuil jamais dépassé';
END;
$$ LANGUAGE plpgsql;
```

Donnez ensuite le résultat de `premier_depassement(5, 5)`, `premier_depassement(5, 50)` et `premier_depassement(1, 5)`.

Exercice 2.4 – Corriger le programme

Chacun des programmes suivants est faux. Indiquez à quel moment le problème se manifeste (au moment de la création de la fonction, ou seulement à l'appel?), expliquez-le, et réécrivez la version

correcte.

- a) *Compte les locations d'un client.*

```
CREATE OR REPLACE FUNCTION compte_locations(p_id INTEGER) RETURNS INTEGER AS $$
DECLARE
    c_r CURSOR FOR SELECT rental_id FROM rental WHERE customer_id = p_id;
    v_r RECORD;
    v_nb INTEGER := 0;
BEGIN
    LOOP
        FETCH c_r INTO v_r;
        EXIT WHEN NOT FOUND;
        v_nb := v_nb + 1;
    END LOOP;
    CLOSE c_r;
    RETURN v_nb;
END;
$$ LANGUAGE plpgsql;
```

- b) *Renvoie la liste des titres de films, séparés par des points-virgules.*

```
CREATE OR REPLACE FUNCTION tous_les_titres() RETURNS TEXT AS $$
DECLARE
    v_res TEXT := '';
BEGIN
    FOR v_f IN SELECT title FROM film ORDER BY title LOOP
        v_res := v_res || v_f.title || ',';
    END LOOP;
    RETURN v_res;
END;
$$ LANGUAGE plpgsql;
```

- c) *Renvoie les titres des films d'une catégorie, une ligne par film.*

```
CREATE OR REPLACE FUNCTION titres_categorie(p_nom category.name%TYPE) RETURNS TEXT
AS $$
DECLARE v_f RECORD;
BEGIN
    FOR v_f IN SELECT f.title
                FROM film f JOIN category ca ON f.category_id = ca.category_id
                WHERE ca.name = p_nom
    LOOP
        RETURN NEXT v_f.title;
    END LOOP;
END;
$$ LANGUAGE plpgsql;
```

Exercice 2.5 – Écrire le programme

- `etat_clients()` : renvoie une table (nom, nombre de locations, nombre de locations en cours) pour **tous** les clients, triée par nom. Utilisez un curseur implicite sur `customer`.
- `premier_film_long(p_duree)` : renvoie le titre du premier film, par ordre alphabétique, dont la durée dépasse `p_duree`, et lève une exception s'il n'en existe aucun. Utilisez un curseur **explicite** et ne lisez qu'une seule ligne.
- * `taux_retour_tous()` : renvoie une ligne de texte par client précisant le taux de retour des locations (nombre de locations rendues / nombre total de locations), de la forme 'Nom : 75.0 %', avec 'Nom : n/a' pour les clients sans aucune location. La gestion du cas particulier doit se faire par un bloc `EXCEPTION` à l'intérieur de la boucle.

Exercice 2.6 – Regard critique sur un programme généré par IA

Cas A – Énoncé : « Écris une fonction PL/pgSQL qui renvoie la liste des noms des clients ayant au moins une location en cours (non rendue). »

Programme proposé par une IA générative

```
CREATE OR REPLACE FUNCTION clients_en_cours() RETURNS SETOF TEXT AS $$
DECLARE
    v_c RECORD;
    v_nb INTEGER;
BEGIN
    FOR v_c IN SELECT customer_id, last_name FROM customer ORDER BY customer_id LOOP
        SELECT COUNT(*) INTO v_nb FROM rental
        WHERE customer_id = v_c.customer_id AND return_date IS NULL;
        IF v_nb = 0 THEN
            EXIT;
        END IF;
        RETURN NEXT v_c.last_name;
    END LOOP;
END;
$$ LANGUAGE plpgsql;
```

Cas B – Énoncé : « Écris une fonction PL/pgSQL qui calcule le montant total payé par un client, en parcourant ses paiements avec un curseur. »

Programme proposé par une IA générative

```
CREATE OR REPLACE FUNCTION total_paye_client(p_customer_id INTEGER) RETURNS NUMERIC
AS $$
DECLARE
    c_pay CURSOR FOR SELECT amount FROM payment WHERE customer_id = p_customer_id;
    v_p RECORD;
    v_total NUMERIC;
BEGIN
    OPEN c_pay;
    LOOP
        FETCH c_pay INTO v_p;
        EXIT WHEN NOT FOUND;
        v_total := v_total + v_p.amount;
    END LOOP;
    CLOSE c_pay;
    RETURN v_total;
END;
$$ LANGUAGE plpgsql;
```

Pour chaque cas : le programme répond-il à l'énoncé ? Que renvoie-t-il réellement sur l'instance de l'annexe A du TD1 ? Proposez une version correcte. Pour le cas B, indiquez de plus si le curseur était justifié.

A Aide-mémoire PL/pgSQL

1. Structure d'une fonction

```
CREATE OR REPLACE FUNCTION nom_fonction(p_param1 INTEGER, p_param2 TEXT)
RETURNS type_de_retour AS $$
DECLARE
    -- déclarations (facultatif)
BEGIN
    -- instructions
    RETURN quelque_chose;
END;
$$ LANGUAGE plpgsql;
```

Les \$\$ délimitent le corps de la fonction (une chaîne de caractères pour PostgreSQL) : à l'intérieur, les apostrophes s'écrivent normalement. Toute instruction se termine par un ;, y compris END IF;, END LOOP; et le END; final. Suppression : DROP FUNCTION nom_fonction(INTEGER, TEXT);

2. Déclarations

```
DECLARE
    v_nb          INTEGER := 0;           -- avec valeur initiale
    v_total       NUMERIC;               -- vaut NULL au départ (piège P2)
    v_titre       film.title%TYPE;       -- même type que la colonne
    v_ligne       film%ROWTYPE;          -- une ligne entière de film
    v_r           RECORD;                -- ligne de structure quelconque
    c_taux        CONSTANT NUMERIC := 0.2; -- constante
```

3. Affectations

```
v_nb := 5;                                -- affectation directe
v_nb := v_nb + 1;
SELECT length INTO v_duree FROM film WHERE film_id = 3;    -- depuis la base
SELECT title, length INTO v_titre, v_duree FROM film WHERE ...; -- plusieurs colonnes
SELECT length INTO STRICT v_duree FROM film WHERE film_id = 3; -- exige 1 ligne
                                exactement
```

Sans STRICT : 0 ligne → variables à NULL, aucune erreur ; plusieurs lignes → la première rencontrée, aucune erreur (P1).

Avec STRICT : 0 ligne → NO_DATA_FOUND ; > 1 ligne → TOO_MANY_ROWS.

FOUND (booléen) indique si la dernière requête a trouvé au moins une ligne – attention, une agrégation en renvoie toujours une (P3).

4. Conditions

```
IF condition THEN ...
ELSIF autre_condition THEN ...
ELSE ...
END IF;

-- expression CASE (renvoie une valeur)
v_libelle := CASE v_rating WHEN 'G' THEN 'tous publics'
                      WHEN 'R' THEN 'adultes'
                      ELSE 'autre' END;
```

Rappel : IF v IS NULL, jamais IF v = NULL (P4).

5. Boucles

```
LOOP ... EXIT WHEN condition; ... END LOOP;    -- boucle générale
WHILE condition LOOP ... END LOOP;             -- boucle tant que
FOR i IN 1..10 LOOP ... END LOOP;              -- compteur (i est implicite ici)
```

```
FOR i IN REVERSE 10..1 LOOP ... END LOOP;
FOR v_r IN SELECT ... LOOP ... END LOOP;          -- curseur implicite (v_r à déclarer)
```

EXIT sort de la boucle, CONTINUE passe à l'itération suivante.

6. Formes de retour

Déclaration	Instructions	Appel
RETURNS INTEGER	RETURN v;	SELECT f(3);
RETURNS SETOF TEXT	RETURN NEXT v; (n fois) ou RETURN QUERY SELECT ...	SELECT * FROM f(3);
RETURNS TABLE(a INT, b TEXT)	a := ...; b := ...; RETURN NEXT; (n fois) ou RETURN QUERY SELECT ...	SELECT * FROM f(3);

Les types renvoyés par RETURN QUERY doivent correspondre *exactement* aux types déclarés (d'où la nécessité dans certains cas de faire des transtypages tel que `f.title::TEXT`). Une fonction non SETOF qui atteint END sans RETURN lève une erreur à l'exécution (P6).

7. Exceptions

```
RAISE NOTICE 'Valeur : %', v_x;          -- message d'information (n'interrompt rien)
RAISE EXCEPTION 'Client % inexistant', p_id; -- interrompt et annule la transaction

BEGIN
    ...
EXCEPTION
    WHEN division_by_zero THEN ...
    WHEN no_data_found THEN ...
    WHEN too_many_rows THEN ...
    WHEN OTHERS THEN RAISE NOTICE 'Erreur : %', SQLERRM;
END;
```

Un bloc BEGIN ... EXCEPTION ... END peut être imbriqué : placé dans une boucle, il permet de poursuivre le parcours malgré l'erreur d'une itération.

8. Curseurs explicites

```
DECLARE
    c_x CURSOR FOR SELECT ... ;          -- curseur sans paramètre
    c_y CURSOR (p_seuil INTEGER) FOR SELECT ... WHERE length > p_seuil;
    v_r RECORD;
BEGIN
    OPEN c_x;                            -- OPEN c_y(120); pour un curseur paramétré
    LOOP
        FETCH c_x INTO v_r;
        EXIT WHEN NOT FOUND;            -- toujours juste après le FETCH (piège P5)
        ...
    END LOOP;
    CLOSE c_x;
```